Лекция 5 Стандарты форматирования сообщений.

Тема: Сервис доставки еды.

| Задание | Выполнение |
|---|---|
| Выберите любое веб-приложение или сервис, которым вы пользуетесь. Определите, какие форматы сообщений оно использует (можно предположить на основе публичной документации API). | Сервис доставки еды (мобильное приложение, веб-сайт, партнёрские рестораны, курьеры). Форматы сообщений: JSON — основной формат публичного REST API (профиль пользователя, меню, заказы, статусы доставки, оплата); YAML — конфигурации (Kubernetes, CI/CD, docker-compose); Protobuf/gRPC — внутренние высоконагруженные микросервисы (трекинг курьеров в реальном времени, ценообразование, маршрутизация); XML — интеграция с legacy-системами ресторанов и платёжными шлюзами; MessagePack/CBOR — мобильные push-уведомления и IoT-трекеры курьеров. |
| Обоснуйте, почему для этого приложения выбран именно такой формат. Какие альтернативы могли бы быть использованы и почему они не подходят? | JSON для публичного REST API: универсальная поддержка во всех языках, человекочитаемость (удобно для отладки, документирования, тестирования через curl/Postman), нативная поддержка в JavaScript (веб-клиент), простота для сторонних разработчиков и партнёров. Альтернативы: XML — избыточен (теги повторяются), медленный (бенчмарк: 1355 мкс против 523 мкс у JSON), сложный парсинг — не подходит для публичного API; Protobuf — нечитаем для человека, требует схемы, неудобен для внешних разработчиков и отладки; YAML — медленный парсинг (5318 мкс), whitespace-чувствителен, не предназначен для API-обмена. Protobuf/gRPC внутри микросервисов: высокая скорость сериализации, компактность, строгая типизация, схемная эволюция, генерация кода, потоковая передача. YAML для конфигураций: читаемость, комментарии, удобство ручного редактирования. |
| Приведите пример данных (например, информация о заказе или профиль пользователя) и представьте их в трёх форматах: JSON, XML, Protocol Buffers (в виде схемы). Сравните размер и читаемость. | Пример — заказ (см. блок ниже). JSON (~270 Б, человекочитаем), XML (~520 Б, читаем, но избыточен из-за тегов), Protobuf (схема .proto) — бинарное представление ~90–110 Б, нечитаемо без инструментов. Сравнение: JSON в 1.5–2 раза компактнее XML; Protobuf в 2–3 раза компактнее JSON и парсится в разы быстрее, но нечитаем. Детальная таблица сравнения — ниже. |
| Опишите, какие преимущества дал бы переход с JSON на Protobuf для выбранного приложения. Какие ограничения могли бы возникнуть? | Преимущества: скорость сериализации/десериализации в разы выше (бенчмарк: 130 мкс против 523 мкс у JSON); компактность (87 КБ против 113 КБ); строгая типизация через схему; схемная эволюция |

| | |
|---|---|
| | (добавление полей без нарушения совместимости); генерация кода для многих языков; поддержка потоковой передачи; меньше нагрузка на сеть и батарею мобильных устройств; ниже задержки в высоконагруженных микросервисах. Ограничения: требуется ведение и версионирование .proto-схем; нечитаем для человека (усложняет отладку, логи, curl-тесты); нужен Schema Registry / контроль версий; сложнее для внешних партнёров и публичного API; риск поломки при неправильном обновлении схемы (отсутствие схемы в Protobuf ломает систему); браузеры не поддерживают нативно — нужны библиотеки (grpc-web). |
| Приведите пример системы, для которой XML остаётся оптимальным выбором. Обоснуйте. | Банковские / корпоративные интеграции и legacy-системы: платёжные шлюзы, SOAP-веб-сервисы, документооборот, интеграция с государственными системами (ГОСТ-форматы обмена, HL7 в здравоохранении, ARINC в авиации, FIX в финансах). Обоснование: XML — зрелый стандарт с XSD-валидацией, поддержкой пространств имён, сложных вложенных структур и атрибутов; широкая распространённость в корпоративной и legacy-среде; строгая проверка документов; обратная совместимость с существующими системами. Переход на JSON/Protobuf потребовал бы миграции всех данных и нарушения совместимости, что экономически нецелесообразно. |

Пример данных к пункту 3: заказ в трех форматах

JSON (~270 байт, человекочитаем):

```
{
  "orderId": 12345,
  "user": {"id": 7, "name": "Иван"},
  "items": [
    {"sku": "A1", "qty": 2, "price": 350.0},
    {"sku": "B2", "qty": 1, "price": 120.5}
  ],
  "total": 820.5,
  "status": "PAID"
}
```

XML (~520 байт, читаем, но избыточен):

```
<order>
  <orderId>12345</orderId>
  <user><id>7</id><name>Иван</name></user>
  <items>
    <item><sku>A1</sku><qty>2</qty><price>350.0</price></item>
    <item><sku>B2</sku><qty>1</qty><price>120.5</price></item>
  </items>
```

```
  <total>820.5</total>
  <status>PAID</status>
</order>
```

## Protobuf (схема .proto; бинарное представление ~90–110 байт, нечитаемо):

```
message User { int32 id = 1; string name = 2; }
message Item { string sku = 1; int32 qty = 2; double price = 3; }
message Order {
  int32 order_id = 1;
  User user = 2;
  repeated Item items = 3;
  double total = 4;
  string status = 5;
}
```

Сравнение форматов к пункту 3.

| Критерий | JSON | XML | Protobuf |
|---|---|---|---|
| Размер примера | ~270 Б | ~520 Б | ~90–110 Б (бинарный) |
| Читаемость | Высокая | Средняя (теги) | Нет |
| Скорость парсинга (бенчмарк) | 523 мкс | 1355 мкс | 130 мкс |
| Схема | Неявная | XSD (опционально) | Обязательна (.proto) |
| Схемная эволюция | Нет | Ограниченная | Да |
| Отладка вручную | Легко | Средне | Сложно |

Итог по теме «Сервис доставки еды»: для публичного API оптимален JSON (универсальность, отладка), для внутренних высоконагруженных микросервисов — Protobuf (скорость, компактность, эволюция), для конфигураций — YAML, а XML остаётся оправданным только в legacy/корпоративных интеграциях (платежи, SOAP, ГОСТ/HL7).